



Programare orientată pe obiecte

1. Colecțiile Java



Limitările tablourilor

- La gestiunea seriilor, seturilor și a grupurilor de date, tablourile nu sunt întotdeauna cea mai bună soluție
- Tablourile nu excelează atunci când datele sunt volatile – în special atunci când mărimea setului de date poate fluctua
 - inserarea unui element necesită glisarea elementelor de deasupra punctului de inserție -> e nevoie de spațiu suplimentar alocat la sfârșit
- Mai general, tablourile expun programatorului de aplicație problemele legate de gestiunea de nivel jos a memoriei
- Dacă cineva dorește să ofere accesul la un tablou privat:
 - poate face accesibil tabloul însuși printr-o metodă accesoare
 - poate furniza o interfață pentru iterare cu metodele: first, next, etc.
 - poate întoarce o copie adâncă a tabloului – lucru foarte ineficient



Colecții versus tablouri

- Lucrul cu API Collections este diferit de lucrul cu tablouri
- Tablourile sunt cu *legare tare la tipuri (strongly typed)*
 - Se specifică tipul elementelor și compilatorul impune tipul la încercarea de asignare de valori la elemente
 - Se pot defini tablouri de elemente de tipuri primitive
- Colecțiile sunt cu *legare slabă la tipuri (weakly typed)*
 - Există o clasă **Vector** și toate elementele sale sunt de tipul **Object** -> toate obiectele de toate tipurile pot fi stocate acolo și e nevoie de forțarea tipului (downcast) elementelor la citire
 - Nu se pot include valori primitive în colecții, deși există o cale de împachetare/învelire (box) a lor – clasele învelitoare (d.e. Integer, Boolean, Double etc.)



Colecții versus tablouri

- Tablourile dau în general viteză mai mare, deoarece reprezintă blocuri de memorie accesibile direct
- Colecțiile sunt obiecte cu metode care trebuie invocate pentru a citi sau scrie elemente
- Colecțiile sunt mult mai ușor de folosit pentru programare de uz general, în special atunci când datele sunt foarte volatile – multe adăugări, ștergeri și modificări directe în timp



Colecții

- ***Colecție Java***: orice clasă care păstrează obiecte și implementează interfața **Collection**
 - De exemplu, clasa **ArrayList<T>** este o clasă colecție Java și implementează toate metodele din interfața **Collection**
 - Colecțiile sunt folosite împreună cu *iteratori*
- Interfața **Collection** este cel mai înalt nivel din cadrul general Java pentru clase colecție
 - Toate clasele colecție tratate aici se află în pachetul **java.util**

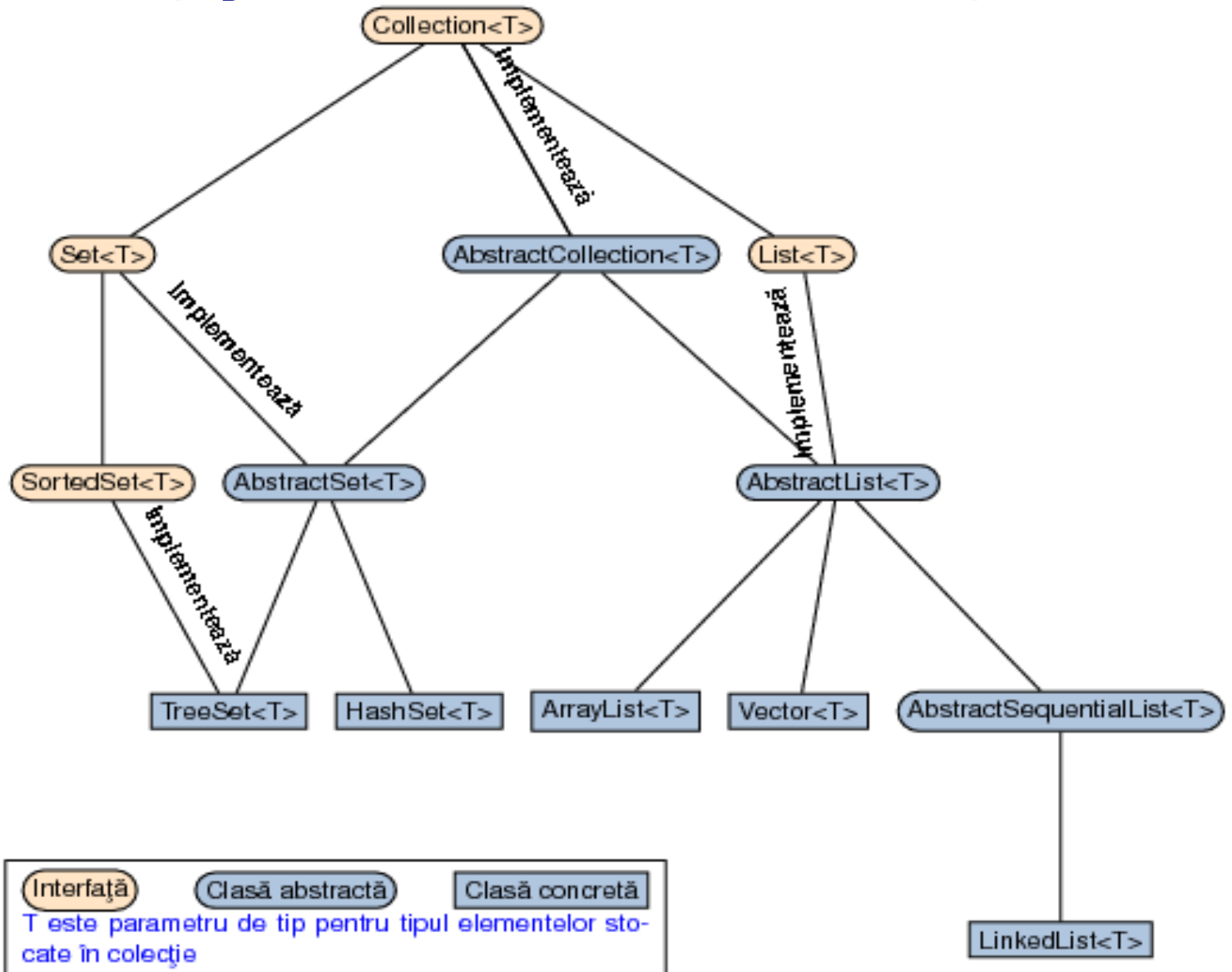


Colecții

- API Collection include:
 - Colecții cum sunt **Vector**, **LinkedList** și **Stack**
 - *Mapări* care indexează valori pe baza cheilor, cum este **HashMap**
 - Variante care asigură că elementele sunt întotdeauna ordonate de un comparator: **TreeSet** și **TreeMap**
 - *Iteratori* care abstractizează abilitatea de a citi și scrie conținutul colecțiilor în bucle și care izolează acea abilitate de implementarea colecției care stă la bază



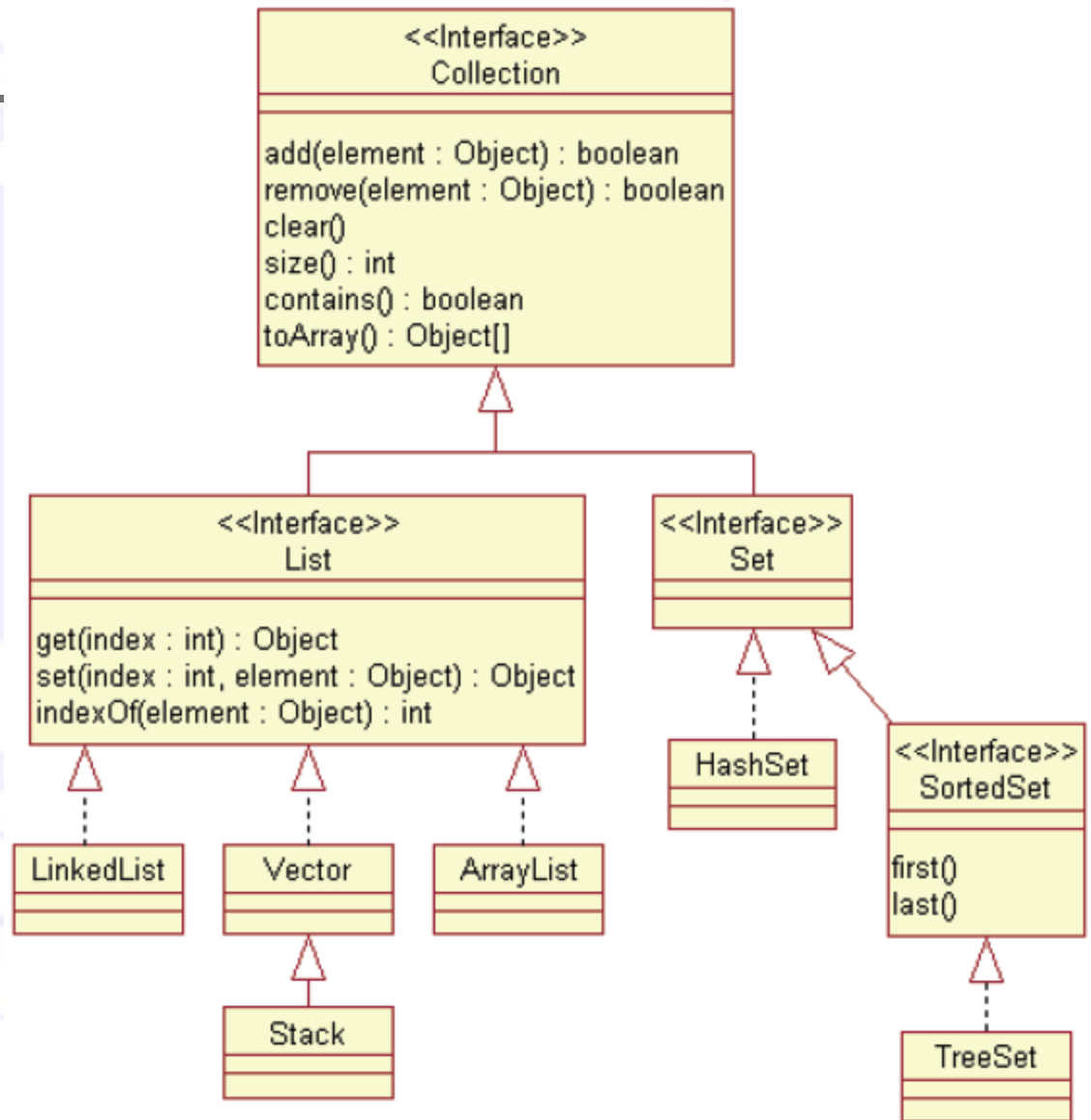
"Peisajul" Collection





"Peisajul" Collection

- Colecțiile ordonate implementează **List**
- Colecțiile care asigură unicitatea elementului implementează **Set**
- Colecțiile sortate implementează **SortedSet**





Caractere de nume nespecificat (wildcards)

- Clasele și interfețele din cadrul general **Collection** pot avea specificații de parametri de tip care nu specifică complet tipul care îl va avea parametrul

- Pentru că ele specifică o gamă largă de tipuri de argumente, ele sunt numite **caractere de nume nespecificat** (*wildcards*)

```
public void method(String arg1, ArrayList<?> arg2)
```

- În exemplul de mai sus, primul argument este de tipul **String**, în timp ce al doilea argument poate fi un **ArrayList<T>** cu orice tip de bază



Caractere de nume nespecificat (wildcards)

- Se poate limita efectul unui caracter de nume nespecificat (wildcard) precizând că tipul trebuie să fie un tip strămoș sau descendent al unei clase sau a unei interfețe
 - Notăția `<? extends String>` specifică faptul că argumentul care va fi folosit trebuie să fie un obiect din orice clasă descendentă a lui `String`
 - Notăția `<? super String>` specifică faptul că argumentul care va fi folosit trebuie să fie un obiect din orice clasă strămoș al lui `String`



Cadrul general `Collection`

- Interfața `Collection<T>` descrie operațiile de bază pe care toate clasele colecție trebuie să le implementeze
- Cum o interfață este un tip, orice metodă poate avea parametri de tipul `Collection<T>`
 - Parametrul respectiv poate fi înlocuit la apel cu orice argument care este un obiect de orice clasă din cadrul general colecție



Interfața Collection

■ Toate colecțiile pot:

- adăuga / elimina elemente
- șterge toate elementele colecției astfel încât rezultă un set vid
- raporta mărimea lor
- converti datele într-un tablou de **Object**

interface Collection

```
{ // lista partiala de metode
```

```
    public int size();
```

```
    public void clear();
```

```
    public Object[] toArray();
```

```
    public boolean add( Object );
```

```
    public boolean remove( Object );
```

```
    public boolean addAll( Collection );
```

```
    public Iterator iterator()
```

- ## ■ Se pot defini proprietăți suplimentare definite de implementarea uneia dintre sub-interfețele

Collection

- colecțiile ordonate implementează **List**
- colecțiile care asigură unicitatea elementelor implementează **Set**
- colecțiile care sortează implementează **SortedSet**



Construirea colecțiilor

- Colecțiile trebuie *create* explicit
 - Greșeală frecventă: declararea unei referințe la un **Vector** sau la o **LinkedList** și presupunerea că obiectul este acolo
- O dată creat obiectul colecție, pur și simplu i se adaugă elemente
 - Folosiți **add** pentru a adăuga un nou element la sfârșit. Atenție că valorile primitive trebuie "învelite" în clase. E.e.

```
Vector<Integer> vec = new Vector<Integer>();  
vec.add(new Integer(5));  
vec.add(new Integer(4));  
System.out.println(vec); // Output: [5, 4]  
int i = ((Integer) vec.elementAt(0)).intValue();
```
 - Folosiți metodele de inserare – definite de subtipuri ale **Collection**
- **remove** (eliminați) un element identificând-ul. Multe subtipuri oferă metode de eliminare bazate pe indecși
- Se poate pune orice obiect Java în orice colecție
 - colecții omogene vs eterogene



Clase colecție concrete

- Clasele **`ArrayList<T>`** și **`Vector<T>`** implementează toate metodele din interfața **`List<T>`** și pot fi folosite așa cum sunt dacă nu e nevoie de metode suplimentare
 - Fiecare dintre ele se poate folosi atunci când este nevoie de o **`List<T>`** cu acces aleatoriu eficient la elemente
- Clasa concretă **`HashSet<T>`** implementează toate metodele din interfața **`Set<T>`** și poate fi folosită așa cum este dacă nu e nevoie de metode suplimentare
 - **`HashSet<T>`** adaugă doar constructori pe lângă metodele din interfață
 - **`HashSet<T>`** este implementată folosind o *tabelă de dispersie*



Clasa Vector

- Oferă accesul aleatoriu la o listă scalară de elemente
 - **Vector** și **ArrayList** au semantica apropiată de un tablou

```
for (int n = 0; n < vec.size(); n++)  
    System.out.println((String) vec.elementAt(n));
```
 - Elementele sunt în ordinea în care au fost adăugate la colecție – nu există sortare implicită
 - Elementele nu trebuie să fie unice în colecție
- Vectorii se comportă cel mai bine la "acces aleator" la elemente – au în spate tablouri
 - punctul slab – ca și la tablouri – inserarea și ștergerea
- Vectorii au capacitate și mărime
 - **size** = mărimea; numărul de elemente aflate curent în colecție
 - capacitate = este dimensiunea tabloului **Object[] elementData** din clasa **Vector**; capacitatea se incrementează automat cu **capacityIncrement** de fiecare dată când **size** devine mai mare decât capacitatea.
 - capacitate $\geq$ **size**



Clasa ArrayList

■ Crearea:

- `new ArrayList()`
- `new ArrayList(int initialCapacity)`

■ Măsurarea:

- `int size()`

■ Stocarea:

- `boolean add(Object o)`
- `boolean add(int index, Object element)`
- `Object set(int index, Object element)`

■ Regăsirea:

- `Object get(int index)`
- `Object remove(int index)`

■ Testarea:

- `boolean isEmpty()`
- `boolean contains(Object elem)`

■ Aflarea poziției (eșec = -1):

- `int indexOf(Object elem)`
- `int lastIndexOf(Object elem)`



Diferențe între `ArrayList<T>` and `Vector<T>`

- Pentru majoritatea scopurilor, `ArrayList<T>` și `Vector<T>` sunt echivalente
 - Clasa `Vector<T>` este mai veche și a trebuit să i se adauge câteva metode pentru a se potrivi în cadrul general colecție
 - Clasa `ArrayList<T>` este mai nouă și a fost creată ca parte a cadrului general colecție Java
 - Clasa `ArrayList<T>` se presupune a fi și mai eficientă decât clasa `Vector<T>`



Exemplu ArrayList

```
import java.util.ArrayList;
import java.util.Iterator;

public class IteratorReferenceDemo
{
    public static void main(String[] args)
    {
        ArrayList<Date> birthdays = new
        ArrayList<Date>( );

        birthdays.add(new Date(90,1, 1));
        birthdays.add(new Date(90,2, 2));
        birthdays.add(new Date(90,3, 3));

        System.out.println("Lista contine:");

        Iterator<Date> i = birthdays.iterator( );
        while (i.hasNext( ))
            System.out.println(i.next( ));
    }
}
```

```
i = birthdays.iterator( );

Date d = null;
System.out.println("Schimbarea
referintelor.");
while (i.hasNext( ))
{
    d = i.next( );
    d.setDate(1);
    d.setMonth(3);
    d.setYear(90);    }

System.out.println("Lista contine
acum:");

i = birthdays.iterator( );
while (i.hasNext( ))
    System.out.println(i.next( ));

System.out.println("Pacaleala 1 aprilie
1990!");
```



Exemplu: Aflarea șirurilor duplicat

```
import java.util.*;
public class FindDups {
    public static void main(String args[]) {
        Set<String> s = new HashSet<String>();
        for (String a : args)
            if (!s.add(a))
                System.out.println("Duplicat: " + a);
        System.out.println(s.size()+" cuvinte distincte: "
                               +s);
    }
}
```

java FindDups i came i saw i learned

Duplicat: i
Duplicat: i
4 cuvinte distincte: [i, learned, saw, came]

- Remarcați faptul că **codul referă întotdeauna Colecția prin tipul interfeței sale (Set)** nu prin tipul implementării (**HashSet**).
- Aceasta este o **practică de programare foarte recomandată** deoarece vă oferă flexibilitatea de a schimba implementările prin simpla schimbare a constructorului.



Exemplu modificat : Aflarea şirurilor duplicat

```
import java.util.*;
public class FindDups2 {
    public static void main(String args[]) {
        Set<String> uniques = new HashSet<String>();
        Set<String> dups = new HashSet<String>();
        for (String a : args)
            if (!uniques.add(a)) dups.add(a);

        // Diferenta de multimi distructiva
        uniques.removeAll(dups);
        System.out.println("Cuvinte unice: " + uniques);
        System.out.println("Cuvinte duplicate: " + dups);
    }
}
```

java FindDups i came i saw i learned

Cuvinte unice: [left, saw, came]

Cuvinte duplicate: [i]



Clase colecție concrete

- Clasa concretă **LinkedList<T>** este derivată din clasa abstractă **AbstractSequentialList<T>**
 - Ar trebui folosită atunci când este nevoie de traversarea secvențială eficientă a unei liste
- Interfața **SortedSet<T>** și clasa concretă **TreeSet<T>** sunt destinate să implementeze interfața **Set<T>** și să ofere regăsirea rapidă a elementelor
 - Implementarea clasei este asemănătoare cu un arbore binar, dar inserarea păstrează echilibrul arborelui



Clasa LinkedList

- Este un alt mijloc de obținere a unei colecții scalare
 - Fiecare element din lista înlănțuită este discret în memorie
 - Elementul conține o referință spre elementul următor și alta spre elementul precedent
- Listele înlănțuite se comportă bine la inserări și ștergeri – nu este nevoie de glisarea elementelor la inserare
 - Se desfac legăturile existente și se formează altele noi
 - Ștergerea implică schimbarea unor legături
- Iterarea este mai lentă
 - Nu se poate căuta aleator, trebuie traversată element cu element



O privire asupra cadrului general Map

- Cadrul general Java *map* tratează colecții de *perechi ordonate*
 - De exemplu, o cheie și valoarea asociată ei
- Obiectele din cadrul general map pot implementa funcții și relații, astfel încât pot fi folosite la construirea claselor pentru baze de date
- Cadrul general map folosește interfața **Map<T>**, clasa **AbstractMap<T>** și clase derivate din clasa **AbstractMap<T>**



Enumerări și iteratori

- Sunt obiecte folosite pentru a parcurge un container.
 - Sunt disponibile pentru unele clase container standard – care implementează interfețele corespunzătoare.
 - Funcționează corect chiar dacă containerul se modifică.
 - Ordinea poate sau nu să fie semnificativă.
- Java are două variațiuni:
 - **Enumeration** (vechi: de la JDK 1.0)
 - **Iterator** (mai nou: de la JDK 1.2)



Enumerări

- Pentru a obține un enumerator **e** pentru containerul **v**:
 - `Enumeration e = v.elements();`
 - **e** este inițializat la începutul listei.
- Pentru a obține primul element și următoarele:
 - `someObject = e.nextElement();`
- Pentru a verifica dacă le-am parcurs pe toate:
 - `e.hasMoreElements();`
- Exemplu:

```
for (Enumeration e = v.elements();  
    e.hasMoreElements(); ) {  
    System.out.println(  
        e.nextElement());  
}
```



Iteratori

- *Iterator*: un obiect folosit la o colecție pentru a furniza accesul secvențial la elementele colecției
 - Acest acces permite examinarea și eventual modificarea elementelor
- Iteratorul impune o ordonare a elementelor colecției chiar dacă colecția în sine nu impune o ordine asupra elementelor pe care le conține
 - În cazul în care colecția impune o ordonare asupra elementelor sale, iteratorul va folosi aceeași ordonare



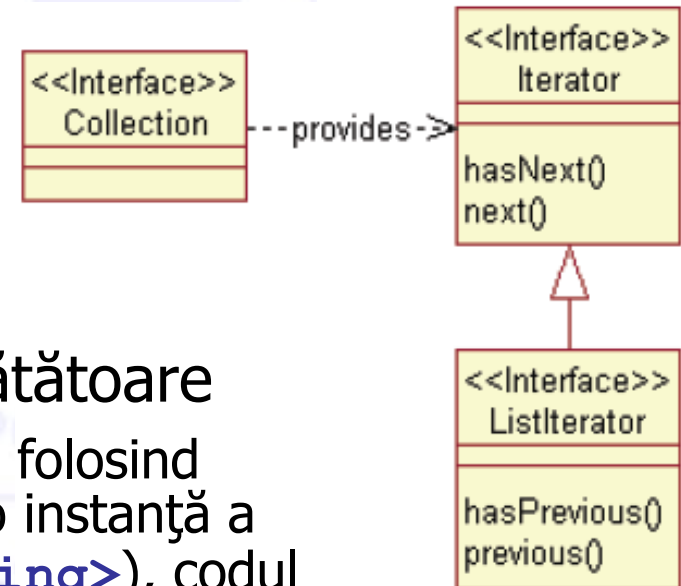
Interfața `Iterator<T>`

- Interfața `Iterator<T>` izolează folosirea unei colecții de clasa colecție în sine

```
interface Iterator
{
    public boolean hasNext();
    public Object next();
    public void remove(); // optional
}
```

- O `Iterator<T>` nu este de sine stătătoare
 - Ea trebuie asociată cu un obiect colecție folosind metoda `iterator`. D.e. Dacă `c` este o instanță a unei clase colecție (d.e., `HashSet<String>`), codul care urmează obține un iterator pentru `c`:

```
Iterator iteratorForC = c.iterator();
```





Folosirea unui iterator cu un obiect `HashSet<T>`

- Un obiect de tipul `HashSet<T>` nu impune nici o ordine asupra elementelor pe care le conține
- Cu toate acestea, un iterator va impune o ordine asupra elementelor din set
 - Aceasta va fi ordinea în care elementele sunt regăsite de `next()`
 - Deși la fiecare rulare a programului ordinea elementelor produse astfel poate fi identică, nu există nici o cerință care să impună acest lucru



Exemplu: iterator peste HashSet<T>

```
import java.util.HashSet;
import java.util.Iterator;
public class HashSetIteratorDemo
{
    public static void main(String[] args)
    {
        HashSet<String> s = new HashSet<String>( );
        s.add("health");
        s.add("love");
        s.add("money");
        System.out.println("The set contains:");
        Iterator<String> i = s.iterator( );
        while (i.hasNext( )) System.out.println(i.next( ));
        i.remove( );
        System.out.println( );
        System.out.println("The set now contains:");
        i = s.iterator( );
        while (i.hasNext( )) System.out.println(i.next( ));
        System.out.println("End of program.");
    }
}
```



Sugestie: Bucle For-Each ca iteratori

- Deși nu este iterator, bucla for-each poate servi în același scop ca un iterator
 - Bucla for-each se poate folosi pentru a parcurge fiecare element al unei colecții
- Buclele for-each pot fi folosite la oricare colecție menționată
- Buclele **for** obișnuite nu pot parcurge elementele dintr-un obiect colecție
 - Spre deosebire de elementele de tablouri, elementele obiectelor colecție nu sunt în mod normal asociate cu indici
- Deși bucla **for** obișnuită nu poate parcurge elementele unei colecții, bucla **for** îmbunătățită poate parcurge elementele unei colecții



Bucula "for each"

- Sintaxa generală a instrucțiunii **for**-each (pentru fiecare) folosită la o colecție este
*for (TipColecție NumeVariabila:NumeColecție)
 Instrucțiune*
- Linia **for**-each de mai sus trebuie citită ca "pentru fiecare **NumeVariabila** din **NumeColecție** execută ceea ce urmează."
 - Remarcați că **NumeVariabila** trebuie declarată în interiorul fiecărei bucle, nu înainte
 - Remarcați, de asemenea, că se folosește simbolul "două puncte" (:) după **NumeVariabila**



Exemplu de buclă For-Each ca iterator

```
import java.util.HashSet;
import java.util.Iterator;
public class ForEachDemo {
    public static void main(String[] args) {
        HashSet<String> s = new HashSet<String>( );
        s.add("health");
        s.add("love");
        s.add("money");
        System.out.println("The set contains:");
        String last = null;
        for (String e : s) {
            last = e;
            System.out.println(e);
        }
        s.remove(last);
        System.out.println( );
        System.out.println("The set now contains:");
        for (String e : s) System.out.println(e);
        System.out.println("End of program.");
    }
}
```



Folosirea genericelor

- Un *tip generic* se definește în termenii unui alt tip pe care îl colectează sau asupra căruia acționează în vreun fel, folosind parantezele unghiulare (<>)

- Exemplu: un **ArrayList<Point>** este un tablou-listă de obiecte **Point** (din pachetul **java.awt**)

```
ArrayList<Point> someList = new ArrayList<Point>();
```

- permite compilatorului să surprindă o eroare de genul:

```
somelist.add(new Dimension(10, 10));
```

- Unui obiect colecție de un anumit tip i se va furniza un iterator specific tipului respectiv

```
Iterator<Point> each = someList.iterator();
```

- Atunci nu mai este necesar să se forțeze conversia la tipul necesar pentru rezultatele metodelor accesoare, d.e.

```
while (each.hasNext())  
    each.next().x = 11;
```



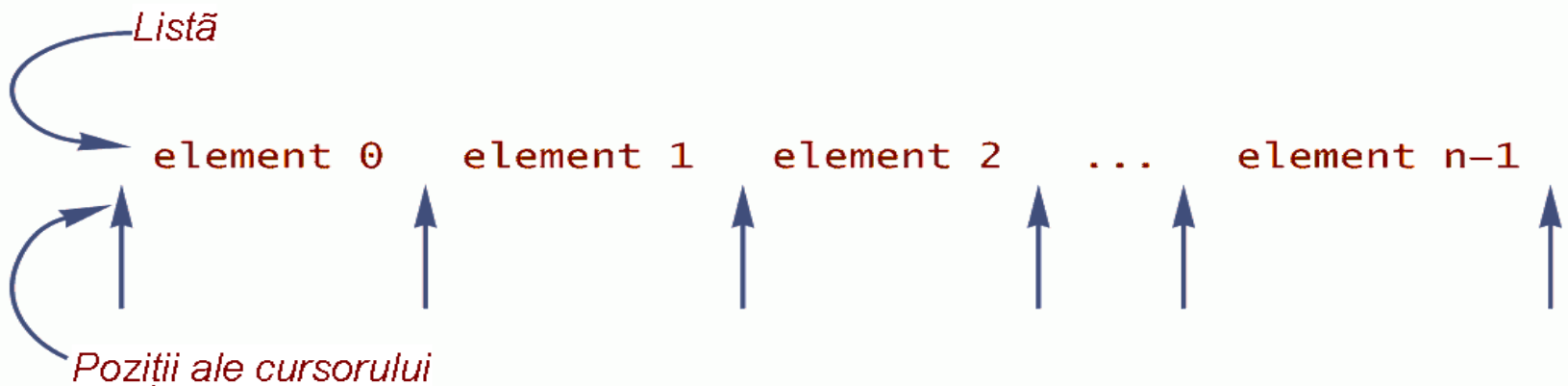
Interfața `ListIterator<T>`

- Interfața `ListIterator<T>` extinde interfața `Iterator<T>` și este menit să lucreze cu colecții care satisfac interfața `List<T>`
 - Un `ListIterator<T>` are toate metodele pe care le are un `Iterator<T>`, plus metode suplimentare
 - Un `ListIterator<T>` se poate deplasa în ambele direcții pe o listă de elemente
 - `ListIterator<T>` are metode cum sunt `set()` și `add()` care se pot folosi la modificarea elementelor



Cursorul `ListIterator<T>`

- Fiecare `ListIterator<T>` are un marcator de poziție numit *cursor*
 - Dacă lista are n elemente, atunci acestea sunt numerotate prin indici de la 0 la $n-1$, dar există $n+1$ poziții ale cursorului
 - La apelul metodei `next()`, se returnează elementul care urmează imediat după cursor, iar cursorul este deplasat înainte cu o poziție
 - La invocarea metodei `previous()` se returnează elementul care urmează imediat înaintea cursorului, iar cursorul este deplasat înapoi cu o poziție de cursor



Poziția inițială implicită a cursorului este cea mai din stânga



Capcană: `next` și `previous` pot întoarce o referință

- Teoretic, atunci când o operație a iteratorului întoarce un element al colecției, el poate returna o copie sau o clona a elementului sau poate returna o referință la element
- Iteratorii pentru clasele colecție standard predefinite, cum sunt `ArrayList<T>` și `HashSet<T>`, returnează de fapt referințe
 - De aceea, modificarea valorii returnate va face modificarea elementului din colecție



Sugestie: Definirea Claselor Iterator proprii

- De obicei nu prea este nevoie de clase `Iterator<T>` sau `ListIterator<T>` definite de programator
- Cea mai simplă și mai utilizată cale pentru a *defini* o clasă colecție este *să o facem o clasă derivată* a uneia dintre clasele colecție de bibliotecă
 - Procedând astfel, metodele `iterator()` și `listIterator()` devin automat disponibile programului
- Dacă o clasă colecție trebuie definită în vreun alt mod, atunci clasa iterator ar trebui definită ca clasă internă (clasă imbricată) în clasa colecție



Sortarea colecțiilor

■ Exemplu:

```
import java.util.*;
class TestSort {
    public static void main(String[] args) {
        ArrayList<String> stuff = new ArrayList<String>(); // #1
        stuff.add("Denver");
        stuff.add("Boulder");
        stuff.add("Vail");
        stuff.add("Aspen");
        stuff.add("Telluride");
        System.out.println("unsorted " + stuff);
        Collections.sort(stuff); // #2
        System.out.println("sorted " + stuff);
    }
}
```

Output:

```
unsorted [Denver, Boulder, Vail, Aspen, Telluride]
sorted [Aspen, Boulder, Denver, Telluride, Vail]
```



Rezumat și studiu în continuare

- Colecții Java
 - tablouri vs colecții
 - API Collections
 - wildcards, generice
 - construirea colecțiilor
 - clase colecție concrete: ArrayList, Vector, LinkedList, HashSet, SortedSet
 - Iteratori
 - Bucle for-each
- De studiat:
 - Lucrarea de laborator "Colecții și tipuri generice"